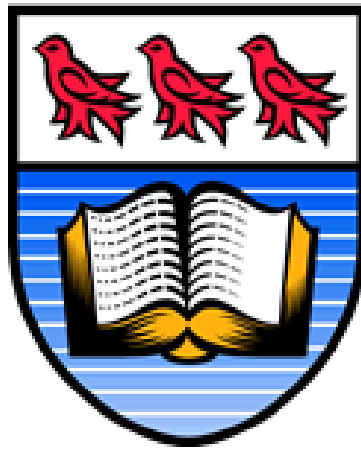


Department of Electrical and Computer
Engineering
University of Victoria



ECE 455 – Real Time Systems

Project 1: Traffic Light System

Submitted: March 5th, 2024

Lab: B01

Hoang Tuan Kiet Vu –

Brandon Radford -

Contents

List of Acronym.....	iii
List of Figures	iii
List of Tables:	iii
Introduction.....	1
Overview	1
Hardware	1
Software	1
Task	1
Queue.....	2
Timer	2
Scheduler	2
Hardware Description	3
Shift Registers Connection.....	3
GPIO Connections.....	4
Software Description	5
TrafficFlowTask.....	5
TrafficLightTask	5
TrafficGeneratorTask	5
SystemDisplayTask.....	5
xTimer	5
System Execution Flow.....	6
Conclusion	8
Appendix.....	9

List of Acronym

TLS: Traffic Light System

LED: Light-emitting diode

GPIO: General-purpose input/output

List of Figures

Figure 1: System Overview Diagram.	2
Figure 2: Shift Register Layout.....	3
Figure 3: Shift Register Pinout.	3
Figure 4: Daisy-chain Connections.....	3
Figure 5: Project connections.....	4
Figure 6: Result of the project during green light.	6
Figure 7: Result of the project after light turn green.	7
Figure 8: Traffic Generating Algorithm.	7
Figure 9: System Display algorithm.	8

List of Tables:

Table 1: GPIOs Layout.	4
-----------------------------	---

Introduction

The objective of this project was to design and implement a Traffic Light System (TLS) using an STM32F4 microcontroller and FreeRTOS software packages. The system required tasks, queues, and schedulers from the FreeRTOS family to control traffic flow in a simulated one-way, one-lane road with an intersection. The TLS incorporated a potentiometer for dynamic traffic flow adjustment, LEDs representing cars, and a traffic light with three LED indicators (green, yellow, and red). The primary goal was to create an efficient traffic control mechanism that adjusts the timing of signals based on real-time traffic flow conditions.

The project also aimed to demonstrate the application of real-time operating system (RTOS) concepts such as multitasking, inter-task communication, and event-driven programming. By leveraging FreeRTOS, the system ensures predictable response times and optimized resource allocation, which are critical for embedded systems managing real-world processes.

Overview

Hardware

The project was implemented using STM32F4 board, involving the following electronics:

- **LEDs:** 19 green LEDs represent the cars moving and 3 LEDs (green, yellow, red) represent the traffic light.
- **Shift Registers (74HC164):** Used to control the LEDs moving when connected.
- **Potentiometer:** Used to simulate the flow of traffic, by assigning a numerical value within the range of the potentiometer and converting it into an adjustable variable in the software.
- **Cables/wires and Breadboard:** Used to connect all of the electronics above to the GPIO pins.
- **Resistors:** Used to regulate the current going into the STM board and other electronics.

Software

Apart from general memory management and error handling pre-defined functions such as:

- **vApplicationMallocFailedHook:** To detect memory allocation failures.
- **vApplicationStackOverflowHook:** To handle stack overflow errors.
- **vApplicationIdleHook:** To monitor available heap memory in an idle state.

The following resources were implemented from the FreeRTOS package:

Task

Specifically, **xTaskCreate** was used multiple times to create the repeated tasks being used throughout the program. Each of the tasks runs independently, allowing parallel execution and efficient control of the system, which was suitable to represent a traffic intersection realistically.

Queue

This module was used to facilitate inter-task communication based on the assigned priority and interrupts. Specifically, the following operations were used:

- **xQueueCreate:** To create the queues.
- **xQueueOverwrite:** To ensure the latest value is stored and discard old, unused values if necessary.
- **xQueuePeek:** To check queue contents without removing them.
- **xQueueReceive:** To retrieve values from the queue.

Timer

This module was used to manage the interval between each execution of the task. It was used dynamically throughout the program to adjust the duration of the traffic lights based on the flow of traffic, creating an adaptive traffic light system. The following operations were used:

- **xTimerCreate:** To create one-shot timers for each light phase.
- **xTimerStart:** To start a timer when a phase begins.
- **xTimerChangePeriod:** To adjust the timer duration dynamically based on the set condition.

Scheduler

The function **vTaskStartScheduler** was used to initialize the FreeRTOS kernel, allowing all the defined tasks to execute based on their priorities/orders.

SYSTEM OVERVIEW DIAGRAM

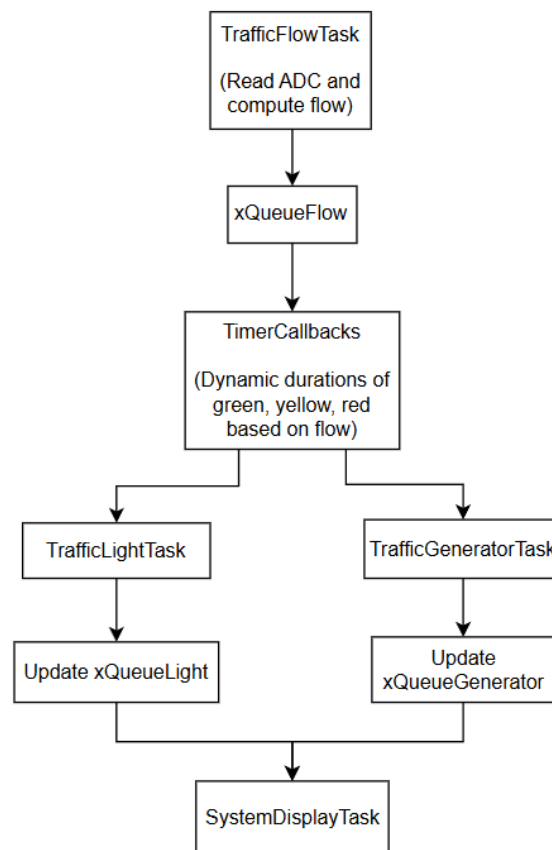


Figure 1: System Overview Diagram.

Hardware Description

Shift Registers Connection

The Shift Register 74HC164 has the following properties:

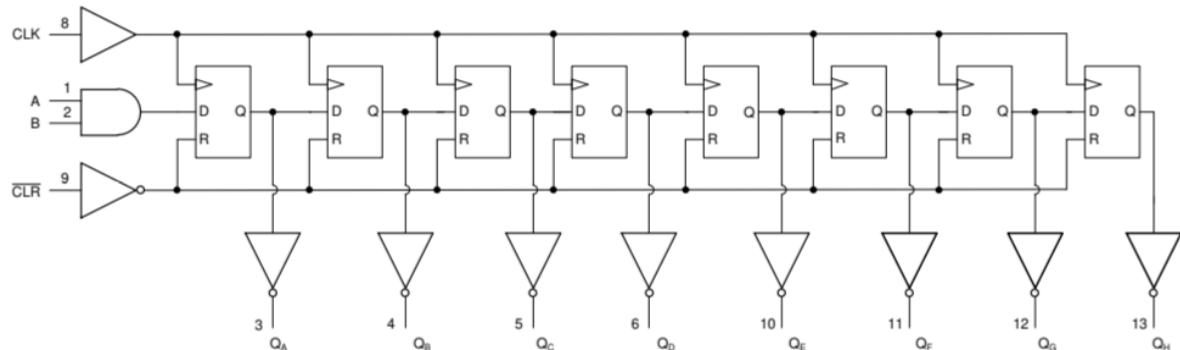
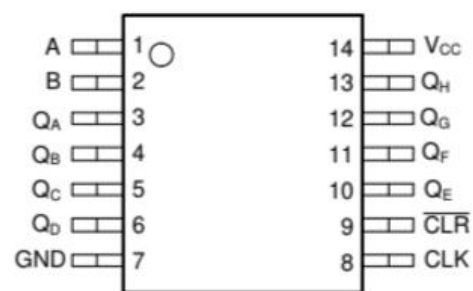
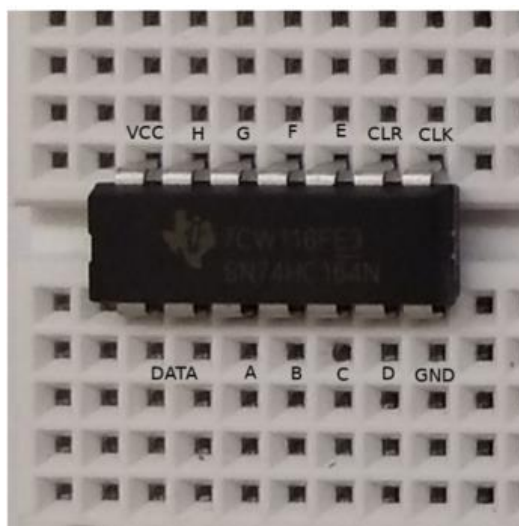


Figure 2: Shift Register Layout.

From the diagram above, A and B ports should be connected together for this project.



**J, D, and N Package
14-Pin CDIP, SOIC, and PDIP
Top View**

Figure 3: Shift Register Pinout.

From the figure and pins layout, in order for three shift registers to operate in series, a special type of connection called a Daisy chain should be implemented.

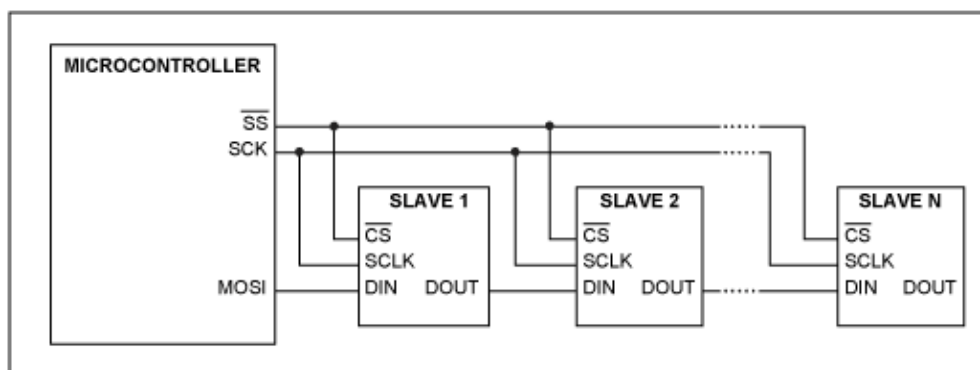


Figure 4: Daisy chain Connections.

From the figure, the Clock and Clear (Reset) ports should be shared in series among the three registers. However, the last input port of the first register should be the first input (data) of the next registers.

GPIO Connections

Below are the required connections as specified in the lab manual. A detailed description of the implementation was also included for reference.

Table 1: GPIOs Layout.

STM32F0	Signal	Notes	Jumper Wires Color
PC0	Green Light	Traffic Light	Green
PC1	Yellow Light		White
PC2	Red Light		Purple
PC3	Potentiometer Input		Yellow
PC6	Shift Register Data	Last input to First input	Orange
PC7	Shift Register Clock	Connected in series	Brown
PC8	Shift Register Reset	Connected in series	Blue
5V			Red
GRD			Black

The circuit was implemented as follows:

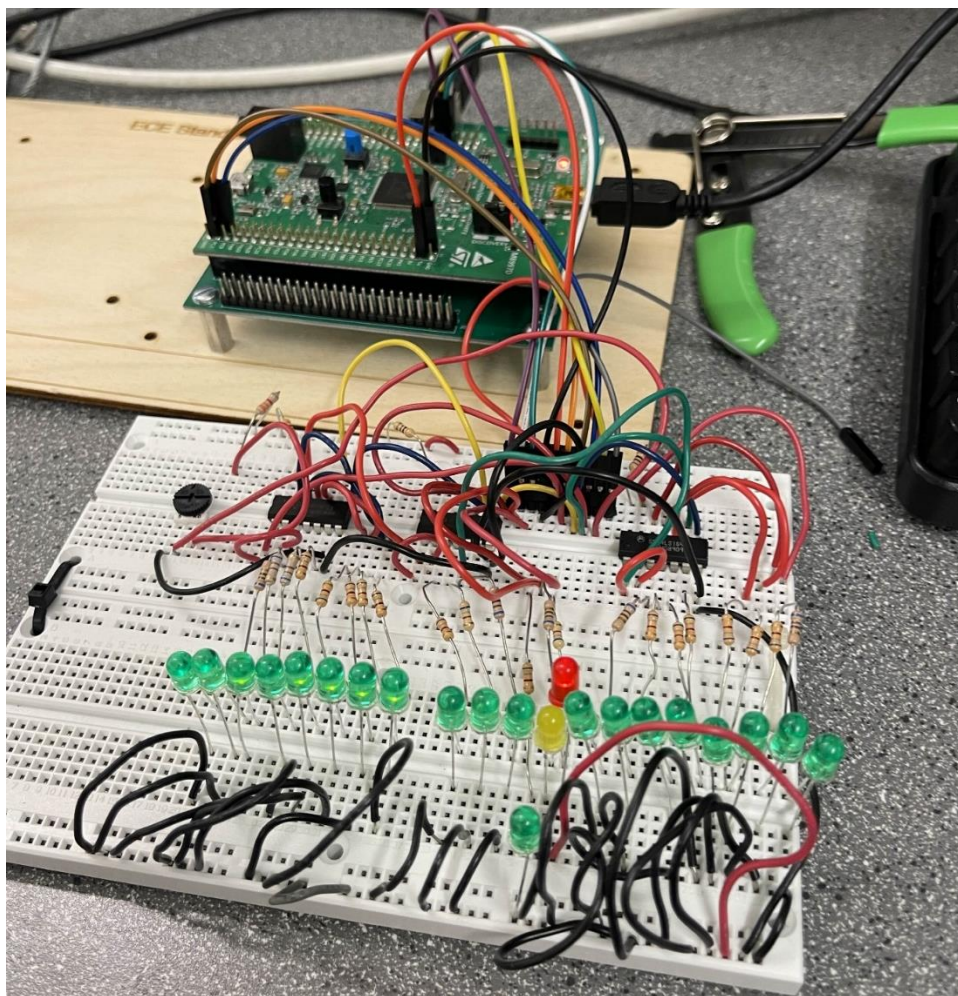


Figure 5: Project connections.

From left to right, the first register was responsible for the first eight green LEDs, representing the cars moving before the traffic light. The second register was responsible for the next three green LEDs representing the car that has crossed the stop line, and the three LEDs used for the traffic lights. The last shift register was used for the last 8 LEDs passed the stop line. It can be seen that the first data input was connected to the last input of the previous shift register as described for Daisy-chain registers.

Software Description

The traffic light control system was constructed using the FreeRTOS package. Overall, the software was responsible for:

1. Controlling the movement of the cars (19 green LEDs).
2. Adjusting the traffic flow based on the potentiometer (green, yellow, and red LEDs).
3. Adjusting the duration of the traffic lights based on the traffic flow.

TrafficFlowTask

Read the ADC value recorded from the potentiometer and convert it into a percentage (0-100%) for easier manipulation. Variable **xQueueFlow** of type **xQueueHandle** was created to communicate the traffic flow rate between each task. The converted flow percentage was updated in the **xQueueFlow** every 100ms and ran indefinitely. This task was created to control and update the traffic flow based on the potentiometer.

TrafficLightTask

The task was used to manage traffic light transitions. It starts with a green light, after each run, variable **xQueueLight** of type **xQueueHandle** was updated to reflect the current light to be displayed at the intersection. This task mostly stays idle, waiting for timer callbacks from the timer callbacks.

TrafficGeneratorTask

It was used to generate new cars and manage current cars' movement. After receiving the traffic flow rate from the first task, the variable **xQueueGenerator** of type **xQueueHandle** was updated to reflect the current presence of cars and stored for the display task. Based on the traffic flow, the task will generate more or fewer cars each run, with a higher flow generating more cars.

SystemDisplayTask

The main task was responsible for updating and displaying the variables in the LEDs. After receiving data from **xQueueLight** for the current traffic light state, and **xQueueGenerator** for new cars, it displays the movement of the cars on the LEDs according to the array of cars by updating the GPIO and Shift Registers. The algorithm also reflects how intersection works in reality, if the cars pass the stop line before the light turn yellow, it will move regardless of the traffic light state.

xTimer

Three timers were created according to the lights at the intersection, with an initial value to be six seconds for both green, and red lights, and three seconds for the yellow light. Both

green and red lights were adjustable later on according to the flow of the traffic, but the yellow light would stay constant for three seconds. By using **xTimerChangePeriod**, the green light will be longer with higher flow, and shorter with less flow, opposite of the red light.

System Execution Flow

FreeRTOS Kernel provides pre-emptive multitasking helps controlling multiple tasks run independently. In this case, the software package was used throughout the project to control the flow of traffic by alternating the LEDs simulating cars moving and control the traffic lights in response to said traffic. Inter-Task communication, specifically **queue** and **timer**, was used to arrange the operation of each task based on their priority and timer. Finally, the HAL on the STM32F0 helps interface the middleware with hardware peripherals, allowing the conversion of ADC for the project to operate smoothly.

Below is a brief description of how the system operates:

1. **Startup:** GPIO & ADC are initialized; Queues & Tasks are created, and the traffic light will start with green
2. **Traffic Flow Monitoring:** The **TrafficFlowTask** will update the current flow of the traffic based on the value recorded from the potentiometer every 100ms.
3. **Car Generation:** The **TrafficGeneratorTask** will generate more or less new cars based on the flow rate.
4. **Traffic Light Control:** The **TrafficLightTask** will start the timer to transition between each light state according to the traffic flow.
5. **Display update:** The **SystemDisplayTask** will update the traffic lights, cars' movement and reflect them in the LEDs on the breadboard.

The results can be observed in the following figures:

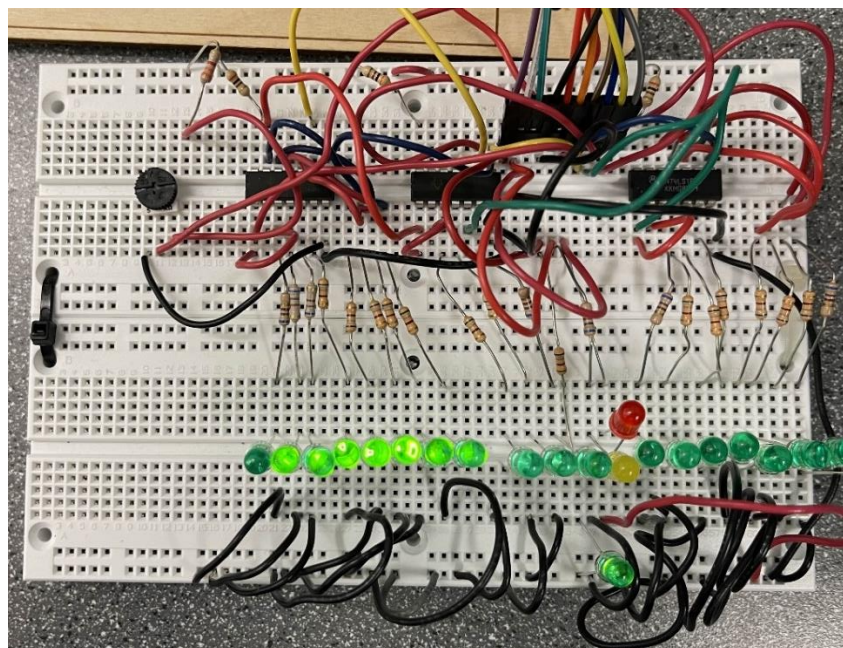


Figure 6: Result of the project during green light.

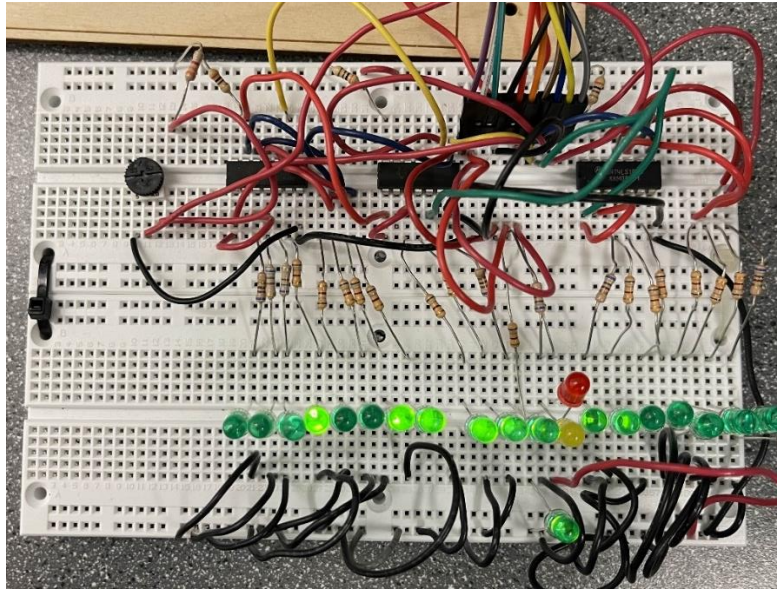


Figure 7: Result of the project after the light turns green.

A more details description of the execution flow will be implemented in the following flowchart.

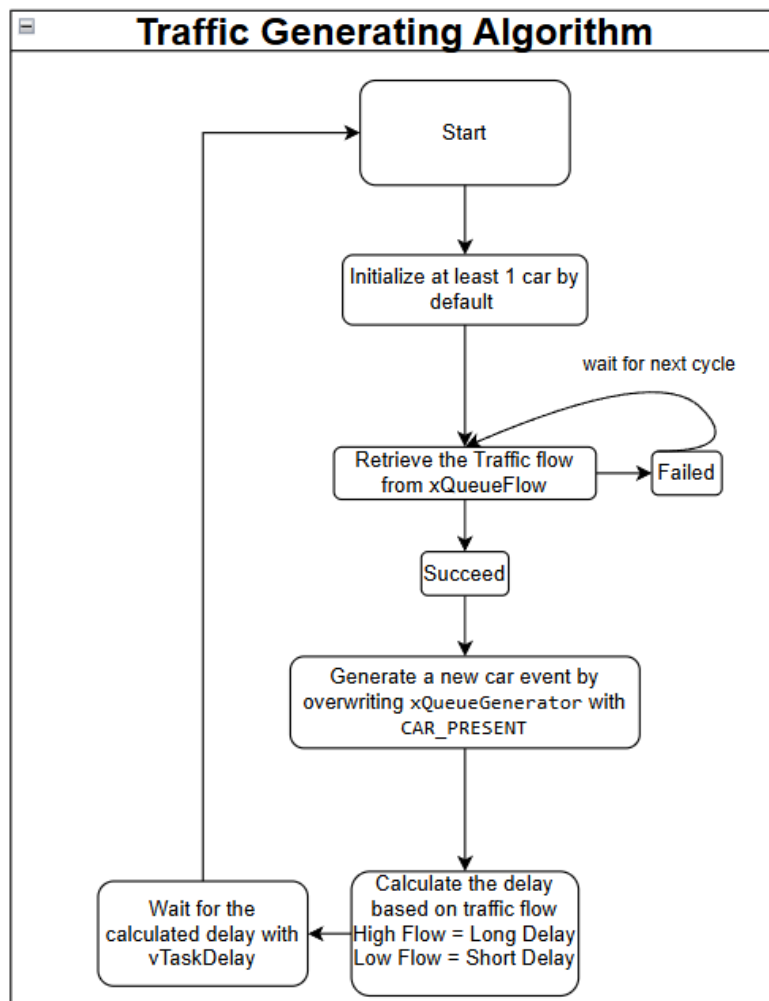


Figure 8: Traffic Generating Algorithm.

SYSTEM DISPLAY ALGORITHM

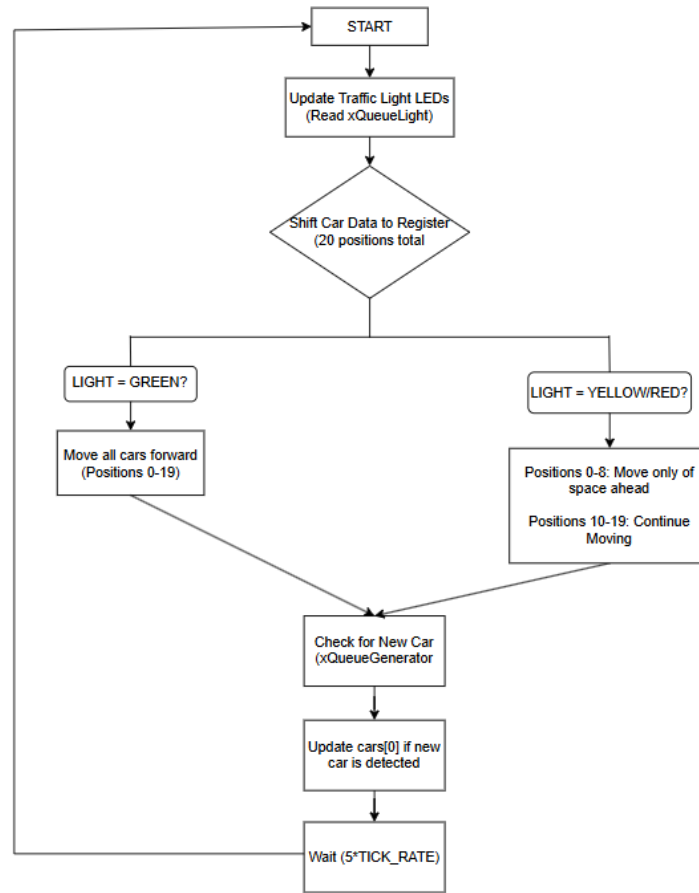


Figure 9: System Display algorithm.

Conclusion

This project successfully implemented a real-time traffic light system using FreeRTOS on an STM32 microcontroller. The use of tasks, queues, and software timers enabled efficient traffic control by dynamically adjusting light durations based on traffic flow conditions. The integration of hardware components such as LEDs, shift registers, and a potentiometer provided a functional and interactive simulation.

The project demonstrated the effectiveness of real-time operating systems in embedded system applications, showcasing the benefits of modular task management and inter-task communication. By using FreeRTOS, the system efficiently managed multiple processes, ensuring responsiveness and predictability.

Future enhancements could include:

- **Reflective Display Intervals:** Currently, the program implemented a constant display rate of 100ms 500ms. However, there was a possibility that the shift register was in the middle of the transition state, but the light was not updated accordingly. This was not a big issue but is a good improvement for seamless traffic light simulation.

- **Better power distribution:** The last 8 LEDs were not fully on and barely noticeable. This could be due to the low power receiving towards the end. Using a smaller value of resistors would possibly help it light brighter.
- **Varying the speed of the cars:** For a more realistic result, each car should not follow the same flow as implemented.

Overall, this project serves as a strong foundation for advanced traffic management systems, demonstrating practical embedded system development and real-time computing techniques.

Appendix

```
// Hoang Tuan Kiet Vu - 
//
// STM32F4 Traffic Light System with FreeRTOS
// This program implements a traffic light control system using FreeRTOS on an
STM32F4 microcontroller.
// The system simulates traffic flow, controls traffic lights, generates cars,
and displays the system state.

/* Standard includes */
#include <stdint.h>      // For standard integer types (uint16_t, etc.)
#include <stdio.h>       // For standard I/O functions
#include "stm32f4_discovery.h" // Board-specific definitions for STM32F4
Discovery board
#include "stm32f4xx_adc.h" // ADC peripheral driver for STM32F4
#include "stm32f4xx_gpio.h" // GPIO peripheral driver for STM32F4

/* FreeRTOS includes */
#include "stm32f4xx.h" // STM32F4 specific definitions
#include "../FreeRTOS_Source/include/FreeRTOS.h" // Core FreeRTOS
functionality
#include "../FreeRTOS_Source/include/queue.h" // FreeRTOS queue API for
inter-task communication
#include "../FreeRTOS_Source/include/semphr.h" // FreeRTOS semaphore API for
synchronization
#include "../FreeRTOS_Source/include/task.h" // FreeRTOS task management
API
#include "../FreeRTOS_Source/include/timers.h" // FreeRTOS software timer
API

/* Traffic Light Definitions */
#define FLOW_MIN 0 // Minimum value for traffic flow sensor (ADC reading)
#define FLOW_MAX 4095 // Maximum value for traffic flow sensor (ADC reading)
Maximum for 12 bit is 4095
#define TICK_RATE pdMS_TO_TICKS(100) // Convert 100ms to FreeRTOS ticks for
task delays

// GPIO pin definitions for the traffic light LEDs and control signals
```

```

#define RED GPIO_Pin_0      // Pin for red traffic light
#define YELLOW GPIO_Pin_1  // Pin for yellow traffic light
#define GREEN GPIO_Pin_2   // Pin for green traffic light
#define POT GPIO_Pin_3     // Pin for potentiometer (analog input for flow
simulation)
#define DATA GPIO_Pin_6   // Pin for shift register data line (for car
display)
#define CLOCK GPIO_Pin_7   // Pin for shift register clock line
#define RESET GPIO_Pin_8   // Pin for shift register reset line
#define CAR_PRESENT 1      // Value indicating a car is present
#define CAR_ABSENT 0      // Value indicating no car is present

/* Function Declarations */
void TrafficFlowTask(void *pvParameters);    // Task to read and process
traffic flow sensor
void TrafficLightTask(void *pvParameters);    // Task to control traffic
light sequence
void TrafficGeneratorTask(void *pvParameters); // Task to generate new cars
based on traffic flow
void SystemDisplayTask(void *pvParameters);    // Task to update visual
display of the system
void TimerCallbackGreen(TimerHandle_t xTimer); // Timer callback for green
light duration
void TimerCallbackYellow(TimerHandle_t xTimer); // Timer callback for yellow
light duration
void TimerCallbackRed(TimerHandle_t xTimer);   // Timer callback for red
light duration
void myGPIOC_Init(void);    // Initialize GPIO Port C pins
void myADC_Init(void);      // Initialize ADC for potentiometer reading
uint16_t ReadADC(void);     // Function to read the current ADC value

/* Global Queues */
xQueueHandle xQueueFlow = NULL;    // Queue to communicate traffic flow
rate between tasks
xQueueHandle xQueueLight = NULL;   // Queue to communicate current traffic
light state
xQueueHandle xQueueGenerator = NULL; // Queue to communicate new car
generation events

/* Timers */
TimerHandle_t xTimerGreen = NULL;   // Timer for green light duration
TimerHandle_t xTimerYellow = NULL;  // Timer for yellow light duration
TimerHandle_t xTimerRed = NULL;     // Timer for red light duration

int main(void)
{
    // Initialize GPIO and ADC hardware
    myGPIOC_Init();

```



```

myADC_Init();

// Create the communication queues with a size of 1 (only the latest value
is needed)
xQueueFlow = xQueueCreate(1, sizeof(int));           // Queue for flow rate
xQueueLight = xQueueCreate(1, sizeof(uint16_t));      // Queue for light
state
xQueueGenerator = xQueueCreate(1, sizeof(int));      // Queue for car
generation

// Only proceed if all queues were created successfully
if (xQueueFlow && xQueueLight && xQueueGenerator)
{
    // Create the FreeRTOS tasks
    xTaskCreate(TrafficFlowTask, "FlowControl", configMINIMAL_STACK_SIZE,
NULL, 1, NULL);
    xTaskCreate(TrafficLightTask, "LightControl",
configMINIMAL_STACK_SIZE, NULL, 1, NULL);
    xTaskCreate(TrafficGeneratorTask, "TrafficGen",
configMINIMAL_STACK_SIZE, NULL, 1, NULL);
    xTaskCreate(SystemDisplayTask, "DisplaySystem",
configMINIMAL_STACK_SIZE, NULL, 1, NULL);

    // Create software timers for traffic light state durations
    // pdFALSE means timers are one-shot (not auto-reload)
    xTimerGreen = xTimerCreate("GreenTimer", pdMS_TO_TICKS(6000), pdFALSE,
0, TimerCallbackGreen); // Go to yellow
    xTimerYellow = xTimerCreate("YellowTimer", pdMS_TO_TICKS(3000),
pdFALSE, 0, TimerCallbackYellow); // go to red
    xTimerRed = xTimerCreate("RedTimer", pdMS_TO_TICKS(6000), pdFALSE, 0,
TimerCallbackRed); // go to green

    // Start the FreeRTOS scheduler - this function should never return
vTaskStartScheduler();
}

// Should never reach here unless there was an error in initialization
for (;;)
}

void myGPIOC_Init(void)
{
    // Enable clock for GPIO Port C
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
    GPIO_InitTypeDef GPIO_InitStructure;

    // Configure pins for traffic lights and shift register as outputs
    GPIO_InitStructure.GPIO_Pin = RED | YELLOW | GREEN | DATA | CLOCK | RESET;

```

```

GPIO_InitStruct.GPIO_Mode = GPIO_Mode_OUT;
GPIO_InitStruct.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOC, &GPIO_InitStruct);

// Configure potentiometer pin as analog input
GPIO_InitStruct.GPIO_Pin = POT;
GPIO_InitStruct.GPIO_Mode = GPIO_Mode_AN; // Analog mode
GPIO_InitStruct.GPIO_PuPd = GPIO_PuPd_NOPULL; // No pull-up/pull-down
GPIO_Init(GPIOC, &GPIO_InitStruct);
}

void myADC_Init(void)
{
    // Enable clock for ADC1 peripheral
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);
    ADC_InitTypeDef ADC_InitStruct;

    // Configure ADC1 settings
    ADC_InitStruct.ADC_ContinuousConvMode = DISABLE; // Single conversion
mode
    ADC_InitStruct.ADC_DataAlign = ADC_DataAlign_Right; // Right-aligned data
    ADC_InitStruct.ADC_Resolution = ADC_Resolution_12b; // 12-bit resolution
(0-4095)
    ADC_InitStruct.ADC_ScanConvMode = DISABLE; // Scan only one channel
    ADC_InitStruct.ADC_ExternalTrigConv = DISABLE; // No external trigger
    ADC_InitStruct.ADC_ExternalTrigConvEdge = DISABLE; // No trigger edge

    ADC_Init(ADC1, &ADC_InitStruct);

    // Enable the ADC
    ADC_Cmd(ADC1, ENABLE);

    // Configure ADC channel 13 (connected to PC3/POT) with 144 cycle sample
time
    ADC-RegularChannelConfig(ADC1, ADC_Channel_13, 1,
ADC_SampleTime_144Cycles);
}

uint16_t ReadADC(void)
{
    // Start ADC conversion
    ADC_SoftwareStartConv(ADC1);

    // Wait until conversion is complete (End Of Conversion flag)
    while (!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));

    // Return the converted value (0-4095)
    return ADC_GetConversionValue(ADC1);
}

```

```

}

void TrafficFlowTask(void *pvParameters)
{
    int flowRate;
    for (;;)
    {
        // Read ADC value and convert to percentage (0-100%) based on defined
        min/max range
        flowRate = (100*((int)ReadADC() - FLOW_MIN) / (FLOW_MAX - FLOW_MIN));

        // Update the flow rate queue with the latest value (overwrite
        previous value)
        if(xQueueOverwrite(xQueueFlow, &flowRate))
        {
            vTaskDelay(TICK_RATE);
        }
    }
}

void TrafficLightTask(void *pvParameters)
{
    // Start the system with green light by starting the green timer
    xTimerStart(xTimerGreen, 0);

    // Update light queue with current state (GREEN)
    uint16_t light = GREEN;
    xQueueOverwrite(xQueueLight, &light);

    // This task initializes the light system then mostly waits
    // The timer callbacks handle the actual light changes
    for (;;)
    {
        vTaskDelay(100*TICK_RATE); // Long delay since work is done by timers
    }
}

void TimerCallbackGreen(TimerHandle_t xTimer)
{
    // at the end of green -> go to yellow state
    // Green light phase is over, start yellow light timer
    xTimerStart(xTimerYellow, 0);

    // Update current light state to YELLOW
    uint16_t light = YELLOW;
    xQueueOverwrite(xQueueLight, &light);
}

void TimerCallbackYellow(TimerHandle_t xTimer)

```



```

{ // At the end of yellow -> go to red state
    int flow;

    // Read current traffic flow rate
    if (xQueuePeek(xQueueFlow, &flow, TICK_RATE) == pdPASS)
    {
        // Set red light duration based on flow rate (higher flow = shorter
red light)
        xTimerChangePeriod(xTimerRed, pdMS_TO_TICKS(10000 - 50*flow), 0);

        // Update current light state to RED
        uint16_t light = RED;
        xQueueOverwrite(xQueueLight, &light);
    }
}

void TimerCallbackRed(TimerHandle_t xTimer)
{ // at the end of red -> go to green state
    int flow;

    // Read current traffic flow rate
    if (xQueuePeek(xQueueFlow, &flow, TICK_RATE) == pdPASS)
    {
        // Set green light duration based on flow rate (higher flow = longer
green light)
        xTimerChangePeriod(xTimerGreen, pdMS_TO_TICKS(5000 + 50*flow), 0);

        // Update current light state to GREEN
        uint16_t light = GREEN;
        xQueueOverwrite(xQueueLight, &light);
    }
}

void TrafficGeneratorTask(void *pvParameters)
{
    int car = CAR_PRESENT; // Always generate a car by default
    int flow;

    for (;;)
    {
        // Read current traffic flow rate
        if (xQueuePeek(xQueueFlow, &flow, TICK_RATE) == pdPASS)
        {
            // Generate a new car
            xQueueOverwrite(xQueueGenerator, &car);

            // Delay between car generation depends on flow rate
            // Higher flow = shorter delay (more frequent cars)

```

```

        // Delay range: 1200ms (flow=80%) to 4000ms (flow=0%)
        vTaskDelay(pdMS_TO_TICKS(4000 - 35*flow));
    }
}

void SystemDisplayTask(void *pvParameters)
{
    uint16_t currentLight;
    int newCar;

    // Array to store car positions (20 positions total)
    // Positions 0-8: Cars before the stop line
    // Position 9: Stop line
    // Positions 10-19: Cars after the stop line
    int cars[20] = {CAR_ABSENT}; // Initialize all positions with CAR_ABSENT
    (0)

    // Initialize the shift register by pulsing RESET
    GPIO_SetBits(GPIOC, RESET);

    for(;;)
    {
        // 1. Update Traffic Light LEDs
        // Get current traffic light state from queue
        if (xQueuePeek(xQueueLight, &currentLight, TICK_RATE) == pdTRUE)
        {
            // Turn off all traffic lights first
            GPIO_ResetBits(GPIOC, RED);
            GPIO_ResetBits(GPIOC, YELLOW);
            GPIO_ResetBits(GPIOC, GREEN);

            // Turn on only the current active light
            GPIO_SetBits(GPIOC, currentLight);
        }

        // 2. Update Shift Register Display with car positions **MAIN ONE**
        for (int i = 20; i > 0; i--)
        {
            // Set DATA pin based on car presence at this position
            if (cars[i] == CAR_PRESENT)
                GPIO_SetBits(GPIOC, DATA);
            else
                GPIO_ResetBits(GPIOC, DATA);

            // Pulse CLOCK to shift in the data bit
            GPIO_SetBits(GPIOC, CLOCK);
            GPIO_ResetBits(GPIOC, CLOCK);
        }
    }
}

```

```

    }

    // 3. Update Car Positions based on Traffic Light State
    if (currentLight == GREEN)
    {
        // On GREEN: All cars move forward one position
        for (int i = 19; i > 0; i--)
            cars[i] = cars[i-1];
/*
* 000100
* 100010
* 010001
* etc etc we just update array here
*/
    }
    else // YELLOW or RED
    {
        // On YELLOW/RED: Cars before stop line don't cross it
        // Only move a car forward if the space ahead is empty
        for (int i = 8; i > 0; i--)
        {
            if (cars[i] == CAR_ABSENT)
            {
                cars[i] = cars[i-1];
                cars[i-1] = CAR_ABSENT;
            }
        }

        // Cars after stop line continue moving regardless of light
        for (int i = 19; i > 9; i--)
        {
            cars[i] = cars[i-1];
            cars[i-1] = CAR_ABSENT;
        }
    }

    // 4. Check for New Cars
    // Try to receive a new car event from the generator queue
    cars[0] = CAR_ABSENT; //
    if (xQueueReceive(xQueueGenerator, &newCar, pdMS_TO_TICKS(100)) ==
pdTRUE)
    {
        // Add a new car at the beginning of the queue if signaled -> set
the array of car
        if (newCar == CAR_PRESENT)
            cars[0] = CAR_PRESENT;
    }

```

```

        // Wait before next display update
        vTaskDelay(5*TICK_RATE);
    }
}

// The following functions are standard FreeRTOS hook functions
// that are called when specific system events occur
void vApplicationMallocFailedHook(void)
{
    // Infinite loop to indicate error - system halts
    for(;;);
}

void vApplicationStackOverflowHook(xTaskHandle pxTask, signed char
*pcTaskName)
{
    (void) pcTaskName;
    (void) pxTask;

    // Infinite loop to indicate error - system halts
    for(;;);
}

void vApplicationIdleHook(void)
{
    volatile size_t xFreeStackSize;
    xFreeStackSize = xPortGetFreeHeapSize();

    if(xFreeStackSize > 100)
    {

    }
}

```